1 Lab 01

LAB 01 - Basic Linux

DEFINITIONS

- **Operating System -** A program that acts as an intermediate medium between user and hardware by managing resources like CPU, memory and storage. It provides a stable environment for users while running complex tasks in the background.
- **Linux -** open source operating system, a unix-like kernel that manages hardware resources and provides a platform for applications. It is stable, secure and has various types.
- **Program vs Process -** Program is an executable file stored on a disk containing a set of instructions (passive entity), but a program is an active entity currently in execution who has its own memory space, stack and PCB.
- **User vs Kernel Mode -** User mode is a restricted space where applications run without direct access to hardware to prevent system crash but kernel mode is a privileged state where the OS has unrestricted access to CPU and hardware for critical tasks like memory management.
- **Mode Switching -** Transition from user mode to kernel mode triggered by system calls, interrupts, exceptions.
- **Commands -** They execute instructions and their format is: <command_name> <space> <options/flags> <space> <arguments>

USER MANAGEMENT

Linux can manage multiple users and proper user management ensures security and integrity. Key files involved in it include:

- `/etc/passwd` - store user account details.
- `/etc/shadow` - store encrypted user passwords.
- `/etc/group` - stores group information.
- `/etc/gshadow` - stores secure group details.

CREATING A USER

- `useradd username` - this creates a user without home directory
- `useradd -m username` - this creates a user with home directory

- `useradd -s /bin/bash username` - this specifies the bash shell otherwise linux will give you a dumber shell like `sh` or `zsh`.

Use these ones when you aren't on debian system or need manual control

- `adduser username` - use this one, the best. It is interactive and automatically creates home directories and shell and also prompts you for password.

MANAGING USER PASSWORD

- To set or change a password name: passwd username.
- You can also enforce policies on passwords:
  - **Expiration -** `chnge -M 90 username`, now password will expire after 90 days.
  - **Lock user account -** `passwd -l username`
  - **Unlock user account -** `passwd -u username`

DELETING USERS

- `userdel username` - deletes the user but keeps their home directory.
- `userdel -r username` - deletes the user and also its home directory.

WORKING WITH GROUPS

- `groupadd groupname` - creating groups.
- `usermod -aG groupname username` - adding users to groups.
- `groups username` - viewing group memberships.
- `usermod -g new_primaryGroup_username` - changing primary group.

FILE MANAGEMENT

- `ls` - lists files and directories in the current directory.
- `stat` - gives detailed meta data of files.
- `file fileName.txt` - determines type of data inside file
- `wc file.txt` - count elements from a file.
- `ls -a` - also lists hidden files and ls /dir = list files in the mentioned directory.
- `cd /path/to/dir` - navigate to mentioned directory.
- `pwd` - prints current directory.
- `mkdir new_folder` - Creates a new directory.
- `rmdir empty_folder` - Removes an empty directory.
- `rm file.txt` - Deletes a file.
- `rm -r folder` - deletes a folder and its contents.
- `cp file1.txt file2.txt` - copies a file.

- `cp -r dir1 dir2` - copies a directory recursively.
- `mv old_name new_name` - renames a file or director.
- `mv name new_path` - moves a file or directory.
- `cat file.txt` – Displays file content.
- `tac file.txt` – Displays file content in reverse order.
- `less file.txt` - opens file with scrolling support.
- `more file.txt` - same as less but only moves forward.
- `head -n 10 file.txt` – Displays the first 10 lines of a file.
- `tail -n 10 file.txt` – Displays the last 10 lines of a file.
- `nano file.txt` – Opens a simple text editor.
- `vi file.txt` - Opens a powerful text editor.
- `echo 'Hello' > file.txt` - writes text to a file, overwriting the previous one.
- `echo 'Hello' >> file.txt` - appends at the end of file instead of overwriting.

FILE PERMISSIONS

Determines who can read, write or execute files or directories. There are 3 level of permissions:

- User - The owner who created the file.
- Group - Users belonging to the assigned group.
- Others - All other users in the system.

Permissions are represented as:

- Read (r or 4) - only view
- Write (w or 2) - can modify
- Execute (x or 1) - Run scripts

To check file permissions, run `ls -l myfile.sh`
Output will be like `-rwxr--r-- 1 user group 1234 Mar 28 10:00 myfile.sh`
Here first - means its regular file and 1 at end means its shortcut. After first - each 3 letters represent permissions of the user then group then others. So here user user has read write execute permissions, group and other has only read permissions.

CHANGING PERMISSIONS

Using Symbolic Mode

- Add permissions with +, remove with - and set with =
- `chmod u+x filename` adds execute for user
- `chmod g-w filename` removes write from group
- `chmod o=r filename` set read only for others
- `chmod u=rwx, g=rx,o=` filename set all permissions for user, only read execute for group and no permissions for others.

Using Numeric (octal) Mode

- Each permission has value: read(4), write(2), execute(1)
- `chmod 755 filename` means each digit represents a set of permissions for diff types. User = 7 means (4+2+1) = read+write+execute and group/others = 5 means (read+execute) = 4+1
- `chmod 642 filename` means user has read+write, group has read and others has only write.

Changing ownership with chown

Owner is usually the user but we can change ownership:

- `chown newuser filename` gives the ownership of the file to the mentioned user.
- `chown newuser:newgroup filename` changes both owner and group.
- `chown :newgroup filename` changes group only.

Changing group ownership with chgrp

- `chgrp newgroup filename` changes group.
- `chgrp -R newgroup filename` changes group recursively.

Umask

- This defines default permissions for new files and directories.
- Current ones can be checked by `umask`.
- To set a new umask: `umask 022`.
- The default is 755 for directories and 644 for files.

PROCESS MANAGEMENT

How processes are monitored, prioritized and terminated.

For Monitoring

- `Ps` - it only shows processes running in your current terminal.
- `Ps aux` - standard snapshot of every process on the system.
- `Top` - A real-time dynamic view of the system which updates every few seconds and sorts processes by CPU usage.
- `Htop` - interactive colorful version of top. It also allows you to kill and scroll with mouse and keys.

For Terminating

- `Kill PID` - sends a terminating signal to process ID to close peacefully.

- `Kill -9 PID` - It sends a kill signal to eliminate the process immediately.
- `Pkill process_name` - kills process wrt name instead of id.

For Prioritizing

- `nice` - starts a new program with specific priority. Eg: `nice -n 10 processname`
- `renice` - changes the priority of an already running process. Eg: `renice +5 -p 1234`
- Above examples: 10 is the priority of new task and 1234 is process id and +5 is priority increment.
- `&` - adding it at the end of command makes the process run in background.
- `jobs` - list all tasks currently running in the background of the current shell.
- `fg job_id` - brings background job to foreground.
- `bg job_id` - sends foreground job to background.
- `ctrl+z` - pauses a running process so you can use the fg or bg command on it.

How to find a process and kill it?

- `Ps aux | grep "program name"` #this finds PID
- `Kill -9 PID`

ARCHIVING AND COMPRESSION

- `tar -cvf archive.tar file1.txt file2.txt` – tar is a common tool for bundling multiple files. Here c means create, v means verbose (show progress) and f means filename. It creates a new archive.
- `tar -xvf archive.tar` – It extracts files from an archive.
- `tar -tvf archive.tar` – list files inside the archive.
- `gzip file.txt` – compresses file and renames to file.txt.gz
- `gunzip file.txt.gz` – decompresses back to original state.
- `zip -r myfiles.zip folder/` – creates a zip file of folders. Recursive flag is mandatory.
- `unzip myfiles.zp` - extract the contents.

SEARCH AND TEXT PROCESSING (grep)

grep is used for searching patterns in files. Its behaviour can be altered by using several flags:

- **-i (ignore case)** searches for a term ignoring case sensitivity. Eg: `grep -i linux filename.txt`

- **-iv (invert match)** display lines that do not contain a specific pattern. Eg: `grep -iv the filename.txt`
- **-c (count)** shows the number of lines that match. Eg: `grep -c python filename.txt.` It can be used with other flags to display count instead of lines like **ivc/ic**.
- **-w (whole word)** matches the pattern as a standalone word only. Eg: `grep -w go filename.txt`
- **-n (line number)** shows the line number where the match was found. Eg: `grep -n security filename.txt`
- **-r (recursive)** searches through all files in current and sub directories. Eg: `grep -r AI folder/`
- **$ (end of line)** searches if only the word is last in the line. Eg: `grep -ivc python$ filename.txt`
- **-f (file)** searches the keywords present in 1 file in another file. Eg: `grep -f words.txt filename.txt`
- **-1 (context)** shows the line where the word was matched and also the line around it to show some context. Eg: `grep -1 encryption filename.txt`
- **-ivc (invert+case-sensitive+count)** count lines not containing the word by ignoring word's case sensitivity.
- **-E (extended Regex)** searches for multiple patterns at once. Eg: `grep -E "Java | Python | C" filename.txt`
- **-q (quiet)** it searches but prints nothing on screen but returns 0 and 1 upon success. So you can pair another command with && which will only execute if the word was searched and || means only execute if it fails. Eg: `grep -q "blockchain" filename.txt && echo "Match found" || echo "No match:`

FOLDER STRUCTURE

Less Significant

- /sbin – system binaries for administrative commands.
- /bin – essential user binaries.
- /lib – shared libraries and kernel modules
- All of them are linked to /usr/bin.

Important System Directories

- /boot – stores files needed for booting the system.
- /usr – contains user installed apps and libraries.
- /var – stores logs, caches and temporary files that change frequently.
- /etc – stores system configuration files.

User and Application Specific Directories

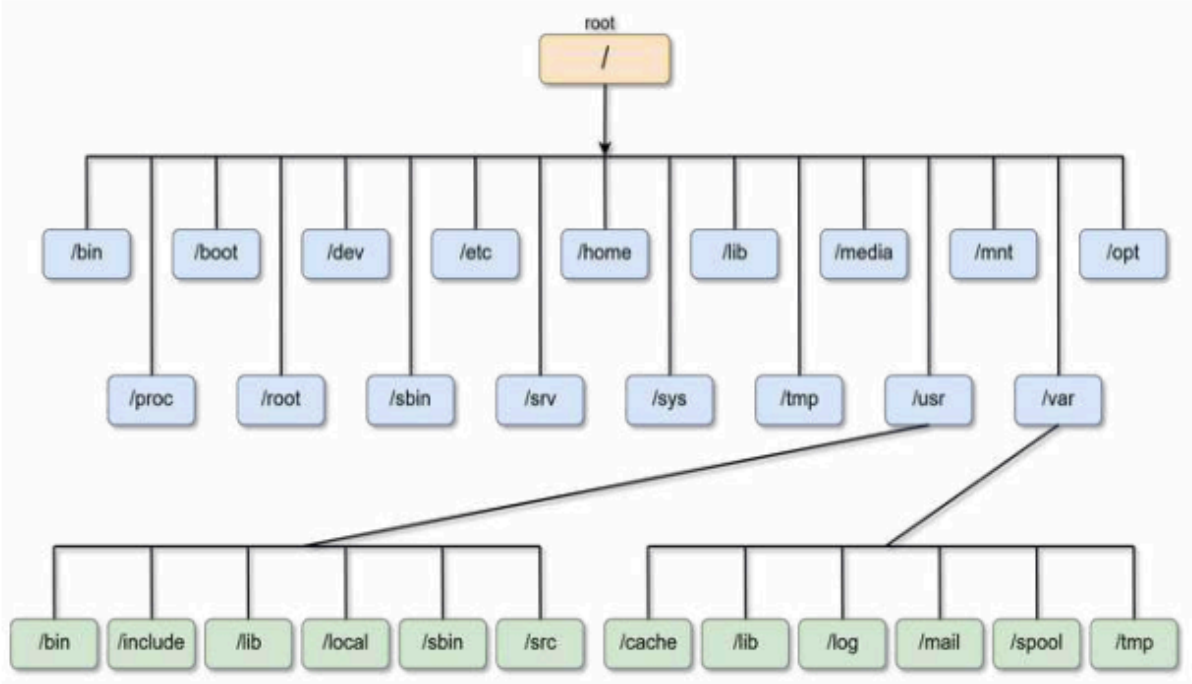
- /home – Default location for user home directories.
- /opt – Used for installing optional third party softwares.
- /srv – Holds data for services like web servers.
- /root – Home directory of the root user.

Temporary and Volatile directories

- /tmp – Temporary files which are cleared on reboot.
- /run – Holds runtime data for processes.
- /proc – Virtual file system for process and system information.
- /sys – Virtual file system for hardware and kernel information.
- /dev – Contains device files like /dev/null or /dev/sda.

Mount Points

- /mnt – Temporary mount point for external file systems.
- /media – Mount point for removable data like USB.
- /data – Your mounted volume from windows (C:/Ubuntu-data)



2 Lab 02

LAB 02 - C/C++ Compilation, Makefile and C Programming

COMPILING C/C++ FILES

For C: `gcc file1.c file2.c … -o output`
For C++: `g++ file.cpp file2.cpp … -o output`
Source files can be other files too like object, assembly, preprocess.
The process is as follows:-
C/C++ files → Preprocessed files(.i) → Assembly Files(.s) → Object Files(.o) → Generation → Final Executable.
This is different from the **compiler process** which is Compiler → Assembler → Linker

When no flag is used, the whole compilation process simply takes place. But:

- -E option is used for running till preprocessed files generation.
- -S option is used for running till Assembly files generation.
- -C option is used for running till Object files generation.

So if we run `gcc main.c -c -o main.o` It will generate object files and to turn them into executables we would run `gcc main.o -o output`

COMMAND LINE ARGUMENTS

They are data passed to the program which is accepted by the main function. Eg: ./output.out 5 2 (we use .out to run via terminal).
In order for the main function to be able to accept parameters we have to change its signature.
`int main (int argC, *int argV[])`
argC is the argument count and argV points to the pointer to our argument. In the case of `./output.out 5 2` The argC=3 because 2 arguments are given and 1 is the file name.
Also these numbers are passed as strings so we have to convert them to integers by `atoi()`.
`cout << atoi(argV[1]) + atoi(argV[2])`

MAKEFILES

When you are making a project you have a lot of files so you won't write each of their names after gcc to compile and make an executable, so for that you create a makefile where we have written the command and everytime we just write **make** to compile. Makefiles provide a way for separate compilation and describe the dependencies among the project files. Standard naming is makefile. `$make filename`
A make file consist of rules like:-

```
target: dependencies
	Commands
```

Target is the name of the file to generate, dependencies are the files on which the target depends. Commands are the actions taken (it must start with tab)
So for a project with 3 files of .c and 2 of .cpp I will everytime have to do `gcc file1.c file2.c file3.c -o output && g++ file1.cpp file2.cpp -o output.` This is bad so we will create a makefile.
Makefile for this

```
output: file1.c file2.c file3.c file1.cpp file2.cpp
	gcc file1.c file2.c file3.c -o output && g++ file1.cpp file2.cpp -o output
```

Now to execute this i.e. compile all files, do `$make output`

Automatic Variables: They are used to refer to specific part of rule components
$@ refers to name of the target of the rule (output)
$< refers to name of the first dependency (file1.c)
$^ refers to all dependencies (all .c and .cpp files)
$? Refers to all the latest dependencies newer then target make

STANDARD TEMPLATE FOR MAKEFILES

```
CC = gcc
CFLAGS = -I.
DEPS = hellomake.h
OBJ = hellomake.o hellofunc.o

%.o: %.c (DEPS)
   $(CC) -c -o $@ $< $(CFLAGS)
hellomake: $(OBJ)
   $(CC) -o $@ $^ $(CFLAGS)
```

Now you can add extra headfiles in DEPS, extra .c files in OBJ (with .o extension). Replace output name from hellomake and also change flags and compiler.

C PROGRAMING REVISION

BASICS

```
#include <stdio.h>
#include <stdbool.h>

int main() {
   //datatypes and variables
   printf("Hello World\n");
   int age=18;
   float marks;
   scanf("%f", &marks);
   printf("Marks: %.2f and Age: %d\n", marks, age);

   //operations
   int a=5,b=10;
   int sum = a+b;
   int diff = 10-sum;
   diff++;
   printf("Sum: %d and Diff: %d\n", sum, diff);

   //if-else
   int number=18;
   if(number>18) printf("Plz vote");
   else if(number==18)    printf("Create NIC then vote");\
   else    printf("Dont vote");
   printf("\n");

   int day = 2;
   switch (day) {
       case 1:
           printf("Monday\n");
           break;
       case 2:
           printf("Tuesday\n");
           break;
       case 3:
           printf("Wednesday\n");
           break;
```

```
        default:
            printf("Invalid day\n");
    }

    //boolean
    bool boolean = true;
    if((age>18 && day==2) || !(boolean)) {
        printf("Either is true\n");
    }

    //loops
    for(int i=0; i<10; i++) {
        printf("i=%d\t",i);
    }

    bool sultanGraet=true;
    while(sultanGraet) {
        printf("\nSultan is Best\n");
        sultanGraet = !sultanGraet;
    }
}
```

FUNCTIONS

```
#include <stdio.h>

int add(int x, int y) {
    return x+y;
}

void greet(char name[]) {
    printf("Greetings %s\n", name);
}

void printArr(int arr[], int size) {
    for(int i=0; i<size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
```

```
}

int main() {
   printf("%d\n", add(2,3));
   greet("Sultan");
   int arr[] = {1,2,3,5};
   printArr(arr,4);
}
```

STRINGS

```
#include <stdio.h>
#include <string.h>

void printNames(char strs[][50], int count) {
   for(int i=0; i<count; i++)  printf("%s\n", strs[i]);
}

int main() {
   char name[50] = "Gemini";
   char input[50];
   printf("Hello, %s\n", name);
   printf("Enter Name: ");
   fgets(input,sizeof(input),stdin);
   printf("Hello: %s\n", input);

   if (strcmp(name,input)==0)  printf("Wow same");
   else    printf("Not same");
   printf("\n");
   strcpy(name,"Claude");
   strcat(name,"-code");
   printf("New NAme: %s\n with letters = %ld\n", name, strlen(name));
   char arrStrs[3][50] = {"Syed", "Sultan", "Ahmed"};
   printNames(arrStrs,3);
}
```

FILING

```
#include <stdio.h>

int main() {
   //1. Declare File ptr
   FILE* fptr;

   //2. Open File in a mode
   fptr = fopen("data.txt","w");   //a is append mode and r is read mode.
also adding + after any mode means update mode too

   //3. Always Check null
   if(!fptr) {
       printf("Error Opening File\n");
       return 1;
   }

   //4. To write in a file instead of console, use fprintf
   fprintf(fptr, "ID: %d, Name: %s", 101, "Gemini");

   //5. Close file
   fclose(fptr);

   //Now we can use fscanf to read from file just like scanf read from
console
   int id;
   char name[20];
   fptr = fopen("data.txt","r");
   //also fscanf returns EOF when file ends
   while(fscanf(fptr, "ID: %d, Name: %s\n", &id, name) != EOF) {
       printf("Read From File --> Name: %s and ID: %d\n", name, id);
   }
   fclose(fptr);
   return 0;
}
```

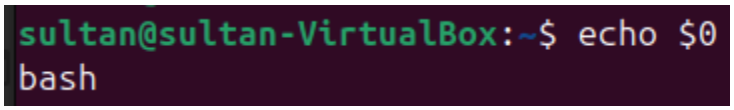
3 Lab 03

LAB 03 - SHELL SCRIPTING

BASIC CONCEPTS

- **What is Linux Shell?** It is a command line interpreter that acts as the interface between user and OS kernel. It takes the commands you type and translates them into the computer's language.
- $ or # means the shell is ready to be prompted.
- The most common shell is Bash (Bourne Again SHell).
- When you put a list of these commands into a file, it is called a **shell script.**

To see which one is your shell, type `echo $SHELL` in terminal to see default path, then `echo $0` to see the name of the current running process which is the shell.

- 

- There are 3 types of shells:
  - **Bourne shell (sh) -** The original one, very fast but basic.
  - **Bourne Again Shell (bash) -** Standard one for linux, has history , tab completion and better scripting.
  - **Z Shell (zsh) -** A modern interactive version of Bash with better plugin support. Default on macOS.
- There are other shells too like ksh,tsh,fish, etc.

SHELL SCRIPTING

Shebang - Every script starts with this special line which tells the bash which interpreter to use. Usually its `#!/bin/bash` but one should verify by `echo $SHELL`.

How to Run a Script?
No compilation needed, but make sure the permission is given which can be done by `chmod +x file.sh`
Then to run the file, do `./file.sh` or `/path/file.sh`
Also when you run a script, it keeps running in terminal but closes if you close terminal, to run script in background we do `nohup ./script.sh` & where `&` tells to run in background and give back you your shell and `nohup` is a signal

CONCEPTS

BASICS

```
#!/bin/bash
#This is a comment

#sending output to terminal
echo "Hello World"

#variables - syntax: varName=value (no space around =) and are accessed
using $
var=150
echo "Value of var = $var"
#we also have constant variables in it
readonly Pi=3.142

#to print newline
echo -e "\n"

#input output
echo "Enter Your Name"
read name
echo "Your name is $name"
#to prompt and input in single line
read -p "Tell your name again: " $name

#random and uid
echo $RANDOM
echo $UID
#common task: find a number between 1 and 100
number=$(( ($RANDOM % 100) + 1 ))
echo "Random Number is $number"
#checking if command successfully ran
ls .
echo $?     #if it return 0. means success else means failure (1-255)
```

OPERATORS

```
#!/bin/bash
var=10+5
echo $var   #bash treat all data as string

#to tell bash that its math, use let
let mathVar=10+10-3
echo $mathVar

#to use arithmetic operators, close them in (()). dont use it on let
variables
echo $mathVar
((a=10*5))
((a+=3))
echo $a

math=80
phy=90
echo "Total: $((math + phy))"
weightedScore = $((math*phy))
echo "Weighted: $weightedScore"
#division not allowed in bash, for it we use bc which is not part of
course.
```

CONDITIONALS

```
#!/bin/bash
marks=50

#after if, write [ condition ] and in next line write then, after it write
your instructions then else then instructions and close it by fi
#you can also use else if concept by writing elif [ condition ] followed
by a then
if [ $marks -gt 40 ]
then
   echo "Passed"
elif [ $marks -eq 40 ]
then
   echo "Draw"
```

```
else
   echo "Failed"
fi

#bash dont have >,< or == etc. -gt is greater, -lt is less than/ -ge and
-le are greater than equal to and less than equal to.
color="Purple"
predefined="Black"
if [ $color != $predefined ]
then
   echo "Your color choice is poor"
else
   echo ""
fi
#for math comparison, we use -eq for == and -ne for != but for string we
use the symbols

#Logical Operators
#same we can use || and && but for it use double bracker [[]]
age=18
iq=100
if [[ $age -ge 18 && $iq -gt 100 ]]
then
   echo "You can Vote"
else
   echo "No voting"
fi
```

ARRAYS

```
#!/bin/bash
array=( 1 2 "Sultan" )
echo $array     #only prints 1st one
echo ${array[2]}
echo ${array[@]}  #prints all
array+=("Ahmed" 18)
echo ${array[*]}    #also prints all

echo "Length of Array:  ${#array[*]}"
```

```
#array slicing - syntax: ${array[source]:start:length}
echo ${array[*]:1}  #start at index 1 but no length specified so full
array
echo ${array[*]:1:3}    #prints 3 elements from index 1

#associative arrays (key-value pairs)
declare -A keyArray     #declared like this, mandotory
keyArray=( [name]="Paul" [age]=20 )
keyArray[city]="NYC"    #adding
echo ${keyArray[name]}
echo ${keyArray[*]} #access all values, also use @
echo ${!keyArray[@]}    #access all keys

#string also array
stringing="Hello"
lengthStr=${#stringing}
upperStr=${stringing^^}
lowerStr=${stringing,,}
echo "String = $stringing and Length is $lengthStr and Uppercase is
$upperStr and Lowercase is $lowerStr"
```

LOOPS

```
#!/bin/bash
for i in 1 2 3 4 5
do
   echo $i
done

for j in "Syed" "Sultan" "Ahmed"
do
   echo $j
done

for num in {1..10}
do
   echo $num
done
```

```
#C-style loop
for ((i=0; i<5; i++))
do
   echo "Iteration $i"
done

#while loop
count=0
num=10
while [ $count -le $num ]
do
   echo "Numbers are $count"
   let count++
done

#like C, we can use break and continue

#until loop - opposite of while
c=1
until [ $c -gt 5 ]
do
   echo "c=$c"
   ((c++))
done

#looping through array
names=( "Sultan" "Hammad" "Ammar" "Ali" )
for individual in "${names[@]}"
do
   echo "Hello, $individual"
done

#infinite loop
# while true
# do
#     echo "Infinite"
#     sleep 2s
# done
```

FUNCTIONS

```
#!/bin/bash
#declaring
function myfunc {
   echo "Greetings"
}

func() {
   echo "Greetings with ()"
}

#calling
myfunc
func

#parameters: arguments are passed but not accessed in function as
parameters instead we use $1, $2 for first parameter second parameterc and
so on
addNums() {
   sum=$(($1+$2))
   echo "Sum is $sum"
}

addNums 10 20

#to return a result, echo it as it is and call fnuction as variable value
getSqr() {
   echo $(($1*$1))
}

square=$(getSqr 5)
echo "Square of 5 = $square"

#all variables by default are global in bash to make someone a private,
use local in function
my_func() {
   local name="Secret"
   echo "Inside: $name"
}
```

```
my_func
echo "Outside: $name"
```

ARGUMENT PASSING

```
#!/bin/bash
#basically arguments can also be passed when calling the file from
terminal. Also accessed like function ones
echo "Script Name: $0"
echo "Total Arguments: $#"

if [ $# -lt 2 ]
then
   echo "Error! Invalid Arguments"
   exit 1
fi

echo "Hello $1 ! You are $2 years old"

#shifting arguments - for long list of arguments, if you use a shift, the
$1 gets discarded and $2 becomes $1 and so on
#eg: processing a list
total=0
echo "Starting with $# arguments"
while [ $# -gt 0 ]
do
   echo "Processing $1"
   ((total++))
   shift
done
echo "Total Processed: $total"

# we can also shift multiple positions like doing shift 2 means $3 becomes
$1 and $4 becomes $2 and both $1,$2 are discarded
```

WHILE LOOP USE CASES

```
#!/bin/bash

#read content from a file
while read -r var   #use -r to avoid escaping backslash
do
    echo $var
done < basics.sh

#read content from a csv file
while IFS="," read -r f1 f2 f3
do
    echo $f1
    echo $f2
    echo $f3
done <data.csv

#another way
cat data.csv | awk awk 'NR>1' |
while IFS="," read -r f1 f2 f3
do
    echo "$f1, $f2, $f3"
done
```

PATH MANIPULATION

```
#!/bin/bash
FILE="~/Documents/Labs/OS/mid-syllabus.txt"
fileName=$(basename "$FILE")
dirName=$(dirname "$FILE")
fullPath=$(realpath "$FILE")

echo "Full String: $File"
echo "File Name: $fileName"
echo "Directory Name: $dirName"
echo "Full Path: $fullPath"

#consider this code
# FILE=$1
```

```
# fileName=$(basename "$FILE")
# cp "$FILE" "/tmp/$fileName.backup"
#you created a script where user run script and give path to a file to
create backup so you can use this manipulation for it
#example: ./backup_script.sh data.txt

#check if file or folder exist
folder="../os-lab3"
if [ -d $folder ]
then
    echo "$folder exists"
else
    echo "$folder dont exists"
fi

filePath="../../mid-syllabus.txt"
if [! -f filePath ]
then
    echo "$filePath exists"
else
    echo "$filePath exists"
fi

#-d -f are is directory and is file flags we used. theres other flags too
#-s for if the the file has a size (not empty). #-e for if the exists
(means true path given). -r for if the file is readable (you have
permission)
```

REDIRECTION

```
#!/bin/bash
## if you dont want to print output to terminal or write it in a file. You
can redirect it to /dev/null
#eg: cd /root &> /dev/null
# since > handles output and 2> handles error. &> handles both

#eg: entering a directory where we dont have permission
set -e
if cd /root &> /dev/null
```

```
then
    echo "I'm in root"
else
    echo "Access Denied but msg is hidden"
fi

set -ex     #e flag stops script if any command fails and x flag prints
every command before executing it
scriptName=$(basename "${0}")   #captures exact script name
echo "Running Script: $scriptName"
logger "Log: $scriptName started by the user $USER"     #sending msg to
system log

ls .    #a working command
mkdir ./newFolder   #now suppose if this folder was existing, then due to
enabling of -e flag, the script would've died here
echo "Finished Successfully"
```

AUTOMATING SCRIPT

You can schedule your scripts to run at any time. Note, in automation always use full absolute paths of scripts.

at

- Use this command if you want your script to run at a time exactly one. Eg: To run your script at 2 am. We will do

```
at 14:00
at> ./script.sh
at> <press ctrl+D to save>
```

- There are different time formats like 14:00, at 10:30pm, at now +1 minute, at tomorrow.

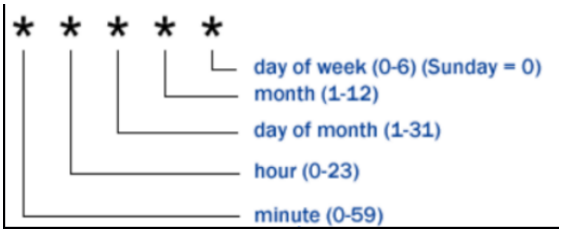
To show list of all scheduled tasks, use `atq`
To remove a scheduled task, use `atrm` [`job_number`]

Crontab

It is for constant scheduling like if you want to run a script every day/hour/week.

When you type `crontab -e` for editing, you will a format like `* * * * * /path/scrpit.sh`

First Star is for minute (0-59)
2nd Star is for hour (0-23)
3rd Star is for day of month (0-31)
4th Star is for month (0-12)
5th Star is for day of week (0-6) where 0 is sunday



Everyday at Midnight - 0 0 * * * /path/script.sh
Every Monday at 8:30 AM - 30 8 * * 1 /path/script.sh
Every 15 minutes - */15 * * * * /path/script.sh

To check existing jobs (already scheduled ones), use `crontab -l` and to add a new job, use `crontab -e`.

What Does This Job Tell? `* * * * cd /home/paul/scripts && ./create file.sh`
This script will run every time by first navigating to the folder specified.

4 Lab 04

LAB 04 - SYSTEM CALLS

It is a mechanism which allows a user program to access kernel level specifications.
Communication with kernel → Transition to kernel mode → Request OS services →Implementation by the kernel → Return values and error.
Following are the windows equivalent system calls:

	Windows	Unix
Process control	CreateProcess()	fork()
	ExitProcess()	exit()
	WaitForSingleObject()	wait()
File management	CreateFile()	open()
	ReadFile()	read()
	WriteFile()	write()
	CloseHandle()	close()
Information maintenance	GetCurrentProcessID()	getpid()
	SetTimer()	alarm()
	Sleep()	sleep()

STRUCTURE

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>

int main () {
   // Declare variables and necessary data structures
   // Make explicit system calls
   // Handle errors if any
   // Perform necessary operations using system calls
   return 0;

}
```

IMPORTANT SYSTEM CALLS

fork()

- It has a return value of more than 0. (PID of child process)
- The parent process invokes the fork() system call to create a child.
- Address spaces are separate but code segments are the same.
- The child executes independently of parent, right from the next line after call.
- The child has a return value of 0 (upon success).
- Both Child and Parent have their own copy of data.

Here's how to create multiple processes with fork:

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
   printf("Inside Parent Process\n");
   pid_t pid1 = fork();     //1 child is created
   if(pid1==0) {
       printf("In Child Process 1. PID=%d and Parent PID=%d\n", getpid(),
getppid());
       exit(0);    //exit doesnt immediately delete all records of this
process, exit status is inside process table. Since its termintaed, its in
zombie state
   }

   pid_t pid2 = fork();    //2 child is created
    if(pid2==0) {
       printf("In Child Process 2. PID=%d and Parent PID=%d\n", getpid(),
getppid());
       exit(0);    //same concept as above, now due to this the parent
might starts executing before its finished so we wait
   }
   int status;
   wait(&status);
   printf("Now back in Parent Process\n");
```

```
}
```

Now the thing is that wait() waits for any child to terminate so it's possible that the parent is completely executed while child2 is still executing, and so is the opposite possible.

```
sultan@sultan-VirtualBox:~/Documents/Labs/OS/os-lab4/practice$ ./2_fork-basic
Inside Parent Process
In Child Process 1. PID=6054 and Parent PID=6053
Now back in Parent Process
In Child Process 2. PID=6055 and Parent PID=6053
sultan@sultan-VirtualBox:~/Documents/Labs/OS/os-lab4/practice$ ./2_fork-basic
Inside Parent Process
In Child Process 1. PID=6108 and Parent PID=6107
In Child Process 2. PID=6109 and Parent PID=6107
Now back in Parent Process
sultan@sultan-VirtualBox:~/Documents/Labs/OS/os-lab4/practice$
```

Now see how fork manages shared code segment with different copies:

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
   int sharerVar=100;  //this code segment is shared across all process.
   printf("Shared Variable Before fork = %d\n", sharerVar);

   pid_t pid=fork();
   if(pid<0) {
       fprintf(stderr,"Fork Failed\n");
       return 1;
   }
   else if(pid==0) {
       sharerVar=200;  //its a seperate copy so modification dont affect
shared value
       printf("I am child (PID=%d) where shared variables = %d\n",
getpid(),sharerVar);
   }
   else {
       //parent
```

```
        printf("I am Parent (PID=%d) where original value is kept, so
shared variable = %d\n",getpid(), sharerVar);
    }
}
```

```
sultan@sultan-VirtualBox:~/Documents/Labs/OS/os-lab4/practice$ ./3_fork-shared_segment
Shared Variable Before fork = 100
I am child (PID=6592) where shared variables = 200
I am Parent (PID=6591) where original value is kept, so shared variable = 100
sultan@sultan-VirtualBox:~/Documents/Labs/OS/os-lab4/practice$
```

wait()

- It is used by the parent process to temporarily suspend its execution.
- It waits for the child to complete its execution.
- Child upon its termination returns exit status which is returned to parent process which after receiving it, resumes execution.
- If you only care about waiting for the child to end and don't need details like exit status, you can use wait(NULL).

waitpid()

- Instead of waiting for all childs, it only waits for the specified child.
- waitpid(pid,status,0);
- Now non-specified childs will stay as zombies in the kernel table.

exit()

- This system call is used to terminate a process immediately from any point in the code, not just the main function.
- Returns 0 (exit(0)) upon success and 1 upon failure (exit(1)).
- It performs cleanup tasks such as closing the opened files and releasing allocated memory.

Example of both wait and exit are in above code

sleep()

- Makes processes become inactive for a given period of time.
- Resumes execution if a time period has passed or interrupt is received.
- In unix, time value is in seconds and in windows, its milliseconds.
- So sleep(2) in linux and sleep(2*1000) in windows.

execl()

- Replaces the current process with a new process. Which means it loads a new program into the memory and starts its execution.
- It has an exit status but cannot return a value upon success (-1 on failure).
- It is one of the variants of various calls in the exec() family.
- In this variant, l stands for list which means you provide a list of arguments for a new program (comma-separated).
- Here you must provide the path to shell commands like /bin/ls
- List must end with null. Eg: `execl(path, arg0, arg1, ..., argN, NULL);`
- In execv(), you provide a vector of arguments and in execlp(), you don't need to provide the path as it searches automatically.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>

int main() {
   pid_t pid1 = fork();
   if(pid1==0) {
       execlp("/bin/ls","ls",NULL);
       exit(1);
   }
   wait(NULL);
}
```

Essential commands like ls,pwd,mkdir are in /bin/ and user defined commands like python,gcc,git are in /usr/bin. System binaries like reboot are in /sbin/.
If you forget, do $ which ls it will give you it.

Zombie Process: A process that has finished execution (called `exit`) but still has an entry in the process table because its parent has not yet "reaped" it using `wait()`.
Orphan Process: A process that is still running but whose parent process has terminated, resulting in the child being "adopted" by the `init` process (PID 1).

To check it: ps aux | grep Z

FILE DESCRIPTORS

They are indexes in a table maintained by the kernel for every process. When a process starts, it automatically opens 3 standard file descriptors:

- **stdin -** For standard input, FD number is 0
- **stdout -** For standard output, FD number is 1
- **stderr -** For standard error, FD number is 2

When a parent process calls fork(), the child receives an exact copy of the parent FD table. The offsets are shared and the file pointers are connected so if a child writes to a file, the pointer moves forward in both parent and child.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <fcntl.h>

int main() {
    int fd1 = open("input.txt", O_RDONLY);
    if (fd1==-1) {
        printf("Failed to open input.txt\n");
        return 1;
    }

    int fd2 = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0777);
    if(fd2==-1) {
        printf("Failed to open output.txt\n");
        return 1;
    }

    char buffer[9];
    read(fd1,buffer,9);
    write(fd2,buffer,9);

    close(fd1);
    close(fd2);
}
```

6 Lab 06

LAB 06 - INTER PROCESS COMMUNICATION

COMMUNICATION BETWEEN PROCESSES

Interprocess communication refers to the mechanisms that allow different processes to communicate and synchronize with each other because processes have separate memory so they need some mechanisms to share data. Pipes allow two processes to communicate where the producer writes to one end and the consumer reads from the other end.

ORDINARY PIPES

Ordinary pipes are also called unnamed pipes. It exists only when the process is running and is only accessible by the process (and its children) that created it. It is created using pipe() system call. Communication flow is unidirectional.

```
sultan@sultan-VirtualBox:~/Documents/Labs/OS/os-lab5$ gcc unnamed_pipes.c -o output
sultan@sultan-VirtualBox:~/Documents/Labs/OS/os-lab5$ ./output
Enter Input MSG: greetings
read greetings
sultan@sultan-VirtualBox:~/Documents/Labs/OS/os-lab5$
```

```
sultan@sultan-VirtualBox:~$ ps -ef | grep output
sultan     23020   20612  0 00:52 pts/0    00:00:00 ./output
sultan     23053   20790  0 00:52 pts/1    00:00:00 grep --color=auto output
sultan@sultan-VirtualBox:~$ ps -ef | grep output
sultan     23020   20612  0 00:52 pts/0    00:00:00 ./output
sultan     23059   23020  0 00:53 pts/0    00:00:00 ./output
sultan     23079   20790  0 00:53 pts/1    00:00:00 grep --color=auto output
sultan@sultan-VirtualBox:~$ ps -ef | grep output
sultan     23113   20790  0 00:53 pts/1    00:00:00 grep --color=auto output
sultan@sultan-VirtualBox:~$
```

See here the process IDs of processes created by unnamed pipes are only available when it is running, after it stops running, they are also gone.
(Use ps -ef | grep output to see running processes).

STEPS

- Define buffer size and read/write ends.
- Create file descriptors (1 for read and 1 for write).
- Read Data

```
char write_msg[BUFFER_SIZE];
```

```
   char read_msg[BUFFER_SIZE];
   int fd[2];  //1 for write end and 1 for read
   pid_t pid;
   printf("Enter Input MSG: ");
   fgets(write_msg,BUFFER_SIZE,stdin);
```

- Create a pipe and child, and check for error

```
if(pipe(fd)==-1) {  //create a pipe
       fprintf(stderr, "Pipe Failed");
       return 1;
   }
   pid = fork();   //create child
   sleep(10.0);
if(pid<0) { //error
       fprintf(stderr, "Pipe Failed");
       return 1;
   }
```

- Now parent will be the write end and child will be the read end.
- The parent process closes the read end and writes to the pipe, then closes the write end.

```
else if(pid>0) { //parent process
       close(fd[READ_END]);
       write(fd[WRITE_END], write_msg, strlen(write_msg)+1);
//writing the msg for which we pass file descriptor and the msg
along with its length
       close(fd[WRITE_END]);
   }
```

- Now the child process closes the write end and reads from the pipe, then closes the read end.

```
else {  //child Process
       close(fd[WRITE_END]);
       read(fd[READ_END], read_msg, BUFFER_SIZE);  //read the msg by
passing fd of read along with msg and buffer size
       printf("read %s", read_msg);    //output the read msg
       close(fd[READ_END]);
   }
   return 0;
```

NAMED PIPES

It exists as a file on the disk even after reboot. Any process with the right permissions can access it. It is created using mkfifo(). They can be bidirectional but that results in race conditions so for two-way communication, create two separate pipes for each direction. Unrelated processes can also communicate with named pipes.

STEPS

In both codes, include the relevant libraries and define path to fifo file.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <string.h>

#define FIFO_FILE "/tmp/myfifo"
```

This myfifo file is created after producer code runs, it exist in /dev/shm/tmp/myfifo

PRODUCER

- Create a file descriptor and a buffer array.
- Create a variable to store the status of read/write.
- Use mkfifo to create a fifo file and give read/write permissions.

```
int fd;
char buffer[BUFSIZ];
ssize_t num_bytes;
mkfifo(FIFO_FILE, 0666);
```

- Open Fifo file in file descriptor as write only and check for error.

```
fd = open(FIFO_FILE,O_WRONLY);
   if (fd==-1) {
       perror("open");
       exit(EXIT_FAILURE);
   }
```

- Keep prompting the producer to write the message and take input.
- Write the input to buffer and verify errors.
- If the input message was “exit”, then stop.

```
while (1) {
        printf("Producer: Enter a message or exit to quit: ");
        fgets(buffer,BUFSIZ,stdin);
        num_bytes = write(fd,buffer,strlen(buffer));
        if(num_bytes==-1) {
            perror("write");
            exit(EXIT_FAILURE);
        }
        if(strncmp(buffer,"exit",4)==0) break;
    }
```

- Close the file descriptors and unlink fifo file.

```
close(fd);
unlink(FIFO_FILE);
```

CONSUMER

- Create a file descriptor and buffer array.
- Create a variable to store the status of read/write.

```
int fd;
    char buffer[BUFSIZ];
    ssize_t num_bytes;
```

- Open fifo file in file descriptor as read only and check for errors.

```
fd = open(FIFO_FILE, O_RDONLY);
    if (fd == -1) {
        perror("open");
        exit(EXIT_FAILURE);
    }
    printf("Consumer: Waiting for messages...\n");
```

- Keep reading the message from the other end of the pipe and store the status.
- If status is 0 it means reading complete, if greater than 0 then keep reading.
- Upon the completion of reading in each iteration, mark the end of line with null terminator.
- Also check if the read message is “exit”, it means reading complete.

```
while (1) {
        num_bytes = read(fd, buffer, BUFSIZ);
        if (num_bytes > 0) {
            buffer[num_bytes] = '\0';
            printf("Consumer: Reeived message: %s", buffer);
```

```
            if (strncmp(buffer, "exit", 4) == 0) {
                break;
            }
        } else if (num_bytes == 0)    break;
    }
```

- Close the file descriptor.

```
close(fd);
    return 0;
```

Consumer reading from producer

```
sultan@sultan-VirtualBox:~/Documents/Labs/OS/os-lab5$ ./consume
Consumer: Waiting for messages...
Consumer: Reeived message: hello world
```

```
sultan@sultan-VirtualBox:~/Documents/Labs/OS/os-lab5$ touch named_pipes-consumer.c
sultan@sultan-VirtualBox:~/Documents/Labs/OS/os-lab5$ gcc  named_pipes-producer.c -o produce
sultan@sultan-VirtualBox:~/Documents/Labs/OS/os-lab5$ gcc  named_pipes-consumer.c -o consume
sultan@sultan-VirtualBox:~/Documents/Labs/OS/os-lab5$ ./produce
Producer: Enter a message or exit to quit: hello world
Producer: Enter a message or exit to quit:
```

SHARED MEMORY

It is the fastest but it needs synchronization to avoid race conditions. Here two or more processes can communicate by sharing a common memory. Changes made by one process can be viewed by another process.

STEPS

In both codes, Include relevant libraries

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <unistd.h>
```

Define name and size of shared memory object.

```
const int SIZE = 4096;  //size of shared memory object
const char *name = "OS";    //name of shared memory object
```

Create file descriptor and pointer of shared memory.

```
int fd; //shared memory file descriptor
```

```
char *ptr;  //ptr to shared memory
```

PRODUCER

- Define the messages to share

```
const char *msg0 = "Hello";
const char *msg1 = "World\n";
```

- Now open the shared object in the file descriptor having either create or read+write mode and permission. Then configure the size of memory.

```
fd = shm_open(name, O_CREAT|O_RDWR, 0666);  //create object
   ftruncate(fd,SIZE);
```

- Map the shared object in file descriptor onto the actual memory

```
   ptr = (char*)mmap(0,SIZE,PROT_READ|PROT_WRITE, MAP_SHARED,fd,0);
```

- Print the message and move forward the pointer to print the next message.

```
sprintf(ptr,"%s",msg0);
   ptr += strlen(msg0);    //in memory we wrote the hello where ptr was pointing now increment ptr by that distance to write further without overwrite
   sprintf(ptr,"%s",msg1);
   ptr += strlen(msg1);
   sleep(10.0);
```

CONSUMER

- After creating the shared object name, size, pointer and file descriptor like producer, open the shared object in the file descriptor in only read+Write mode and map it onto the actual memory so it automatically reads.

```
   fd = shm_open(name,O_RDWR,0666);
   ptr = (char*)mmap(0,SIZE,PROT_READ|PROT_WRITE, MAP_SHARED,fd,0);
```

- Print the read messages and unlink the shared memory.

```
printf("%s", (char*)ptr);
   shm_unlink(name);
   return 0;
```